

Modern C++

Trainer: Nilesh Ghule

Clearer and More Type-Safe Language Features

- Good code communicates intent and makes accidental misuse harder
  - auto
  - range-based for loops
  - initializer_list
  - nullptr
  - console IO
  - scoped enums,
  - type aliases,
  - structured bindings,
  - if/switch initializers,
  - constexpr,
  - inline,
  - inline constexpr,
  - designated initializers

Use auto When the Type Is Clear

- auto asks the compiler to deduce a variable's type from its initializer
- The type is determined at compile time—auto does not make C++ dynamically typed
- Useful when type names are long: iterators, library return values, templates
- Avoid auto when the initializer hides a conversion or the intended type is not obvious.
- C++14 allows auto for deduced return types (of functions).
- C++20 allows auto for function args aka abbreviated function template.

```
auto count = 42;        // int
auto average = 86.5;    // double

auto ratio = 1;         // int, not double

// Better if floating-point is intended:
double ratio = 1.0;
// or
auto ratio = 1.0;       // double
```

Nilesh Ghule

auto, const, and References

- When auto deduces a value type, top-level const and reference qualifiers are not preserved
- auto value = total; creates a copy
- auto& value = total; declares a reference to the existing object
- const auto& provides read-only access without copying

```
int total = 10;
const int& totalAlias = total;

auto copy = totalAlias;        // int: separate copy
const auto& readOnly = total; // const int&: read-only reference

// copy can change without affecting total
// readOnly refers to total; cannot modify total through it
```

Range-Based for Loops

for(init ; cond ; modfn)
{ ... }

- Works with arrays and standard-library containers
- Avoids manual iterator or index management when processing every element
- The loop variable is initialized from each element

```
int scores[]{78, 56, 91};

for (int score : scores)
    std::cout << score << '\n';
```

range based for loop
→ loop repeated for each elem in collection.

... 78, 56, 91

- Range based for loops internally use STL iterators. We will deep dive internals later in the course.

Choose the Right Loop Variable

- for (auto value : scores): creates a copy for each element
  - Appropriate for small built-in types like int; changing the copy does not change the element
- for (const auto& value : scores): read-only reference
  - Good default when inspecting elements, especially expensive-to-copy objects
- for (auto& value : scores): modifiable reference
  - Assignments to value affect the element in the array or container
- Do not apply the same form automatically—decide based on intent

```
for (auto value : scores)          { /* local copy */ }
for (const auto& value : scores) { /* read-only reference */ }
for (auto& value : scores)         { /* reference; may modify */ }
```

auto and Braces

- In C++17 and later, these deduce different types:
  - auto one = {1}; deduces std::initializer_list<int>
  - auto two{1}; deduces int
- Useful when reading existing code,
  but prefer simple initializers like auto two = 1;

```
auto one = {1}; // std::initializer_list<int>
auto two{1};    // int

auto three = 1; // int (clearer)
```

Nilesh Ghule

std::initializer_list (C++11)

- Provides a lightweight way to initialize objects with a list of values using {}
- All elements must have the same type.
- Elements are read-only (const).
- Useful for constructors and functions accepting variable numbers of values.
- std::vector<int> v{1, 2, 3}; uses an initializer_list constructor when applicable.
- initializer_list is a lightweight proxy to a fixed-size array of constant elements; it does not own a dynamically resizable collection.

all int or all floats ...

```
//#include <initializer_list>
void print(initializer_list<int> nums) {
    for (auto n : nums)
        cout << n << " ";
}
int main() {
    print({10, 20, 30}); // 10 20 30
}
```

auto il = {1,2,3};

vector<int> v({1,2,3});

init list

Nilesh Ghule

Prefer nullptr for Null Pointers

- nullptr is a keyword for the null pointer value
- nullptr has the dedicated type std::nullptr_t and is intended to convert to pointer types
- 0 is an integer literal; NULL is a legacy macro whose definition can vary
- Older forms can be surprising in overloaded function calls
- A null pointer is not the same as an uninitialized pointer—always initialize pointers

```
#include <iostream>

void show(int)         { std::cout << "integer\n"; }
void show(const char*) { std::cout << "pointer\n"; }

int main() {
    show(0);        // Selects show(int)
    show(nullptr); // Selects show(const char*)
}
```

Console Output with std::cout

- Use std::cout from <iostream> for simple console output
- Well supported and sufficient for the first examples
- C++23 adds std::print and std::println in <print>—optional when toolset supports them

```
#include <iostream>

int main() {
    std::cout << "Modern C++ starts with clear intent.\n";
}
```

C++23 Formatted Output

- std::print and std::println use replacement fields {} instead of insertion operators
- std::print writes exactly the formatted text; std::println appends a newline
- Format specifiers can control width, alignment, padding, and numeric presentation (octal, dec, hex, bin)
- Requires C++23 mode and toolset support

```
#include <print>
#include <string>

int main() {
    const std::string course{"Modern C++"};
    const int day{1};

    std::print("Welcome to {} - Day {}\n", course, day);
    std::println("Welcome to {} - Day {}", course, day);
}
```

Nilesh Ghule

Format Specifiers (C++23)

- Syntax: {[argument-id]:[fill][align][width][.precision][type]}
  - align: < left, ^ center, > right
  - type: d (decimal), x (hex), f (fixed), e (scientific), etc.
- To print literal braces, double them: {{ and }}

```
std::println("Name: {}, Score: {}", name, score);
std::println("Score with leading zeroes: {:03}", score);
std::println("{:<10} {:>5}", name, score);
std::println("Use {{}} for a placeholder"); // prints: Use {} for a placeholder
```

C++23 Formatted Output

Format specifier	Example	Output	Description
{}	std::println("{}", 42);	42	Default formatting
{:d}	std::println("{:d}", 42);	42	Decimal integer
{:x}	std::println("{:x}", 255);	ff	Lowercase hexadecimal
{:X}	std::println("{:X}", 255);	FF	Uppercase hexadecimal
{:b}	std::println("{:b}", 10);	1010	Binary
{:o}	std::println("{:o}", 8);	10	Octal
{:.2f}	std::println("{:.2f}", 3.14159);	3.14	Fixed-point, 2 decimal places
{:.3f}	std::println("{:.3f}", 3.14159);	3.142	Fixed-point, 3 decimal places
{:e}	std::println("{:e}", 1234.0);	1.234000e+03	Scientific notation
{:g}	std::println("{:g}", 1234.0);	1234	General numeric format
{:08d}	std::println("{:08d}", 42);	00000042	Integer padded with zeros to width 8

C++23 Formatted Output

Format specifier	Example	Output	Description
{:<10}	std::println("{:<10}", "C++");	C++ followed by 7 spaces	Left-align in width 10
{:^10}	std::println("{:^10}", "C++");	3 spaces, C++, 4 spaces	Center-align in width 10
{:>10}	std::println("{:>10}", "C++");	7 spaces followed by C++	Right-align in width 10
{:*>8}	std::println("{:*>8}", 42);	******42	Pad with *, right-aligned
{:010.2f}	std::println("{:010.2f}", 3.14);	000003.14	Width 10, zero-padded, 2 decimal places
{{ and }}	std::println("{{hello}}");	{hello}	Print literal braces

constexpr: Compile-Time Constants

- constexpr (C++11) indicates a variable or function can participate in constant expressions
- const means not modifiable; constexpr requires compile-time initialization
- A constexpr variable is also const, but not every const is constexpr
- Use const for runtime-initialized non-modifiable values; use constexpr for true compile-time constants

must be solved at compile time

→ #define PI 3.14

```
const int x = 10;
constexpr int y = 20;

int input;
std::cin >> input;

const int a = input;     // Valid: runtime initialization
constexpr int b = input; // Error: input is not a constant expression
```

constexpr Functions and static_assert

- constexpr functions can be evaluated at compile time when required
- They can also be called at runtime with runtime arguments
- static_assert checks a condition during compilation
- Use static_assert for compile-time assumptions.

```
constexpr int square(int n) { return n * n; }

constexpr int result = square(5);
static_assert(result == 25);

int value;
std::cin >> value;
int runtimeResult = square(value); // evaluated at runtime

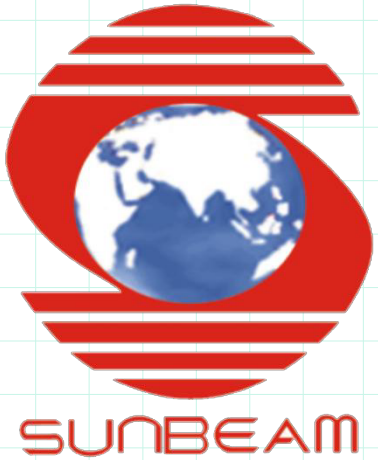
constexpr int BufferSize = 1024;
static_assert(BufferSize > 0, "Buffer size must be positive");
```

Nilesh Ghule

constexpr Evolution and Common Mistakes

- C++11: introduced with strict function-body restrictions
  - Function body generally allowed only a single return statement.
- C++14: relaxed bodies (local variables, if, loops, conditionals)
  - Allows mutation of local variables in compile time evalations
- C++17: more constant-expression capabilities, if constexpr
- C++20/23: further expansion consteval
- Mistakes: assuming const means constexpr; assuming every constexpr call runs at compile time

Nilesh Ghule



Thank You!

Nilesh Ghule

<nilesh@sunbeaminfo.com>